



Module Details

Module Code	UFCFK1-15-0
Module Title	Fundamentals of Data Science
Module Tutors	Saurav Gautam
Year	2025-2026
Component/Element Number	PSA/Bi-weekly assignment/Regular
Weighting	10%

Dates

Submission Date	17-April-2026
Submission Place	Blackboard
Submission Time	23:59
Submission Notes	Submit Gitlab URL

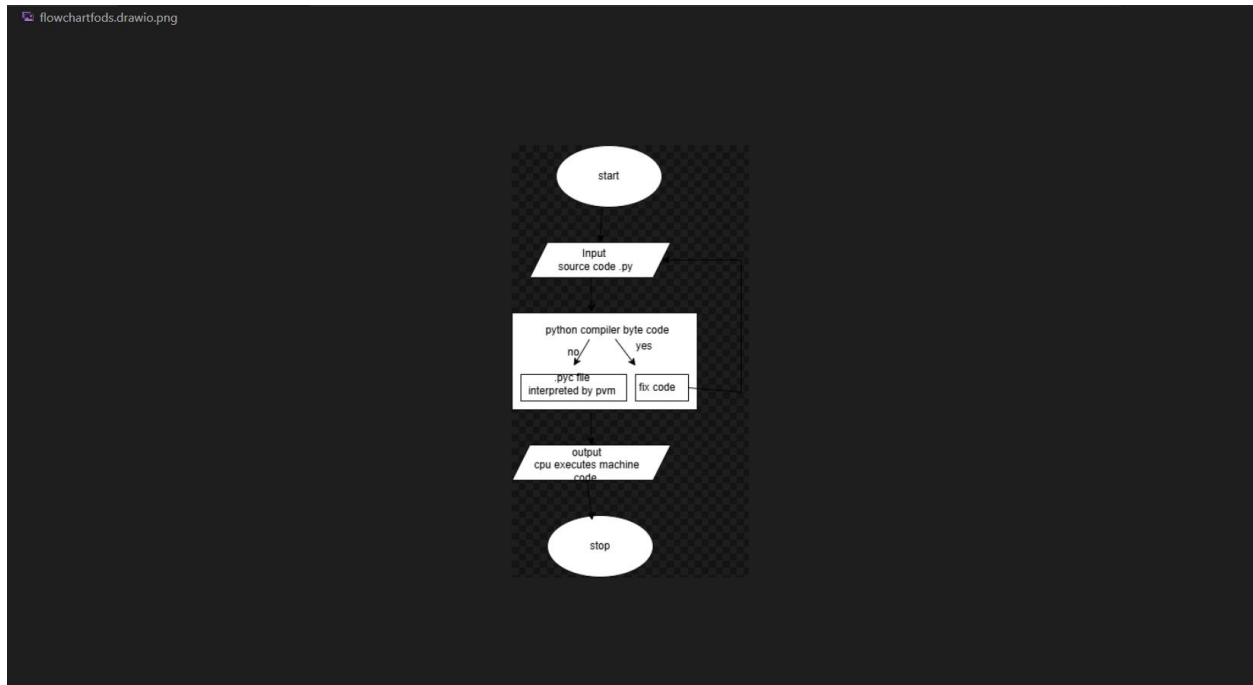
Submitted By: Jonisha Ghimire

Level: 3

Section: A

Student ID: 26002325

1. Create a simple flowchart or diagram to explain the working of Python. The diagram should clearly include input, processing, and output stages. **Marks [10]**



2. Write a program to take **two integer inputs** from the user and display the results of the following operations in a clean format: **Marks [2*5]**
 - I. Addition
 - II. Multiplication
 - III. Division
 - IV. Modulus (remainder)
 - V. Exponentiation

The program takes two numbers as input and perform addition, multiplication, division, modulus, and exponentiation. Also it displays result in readable format.

Output:

```
python -u "c:\Users\userr\OneDrive\Desktop\fods1\q2.py"
enter the first number:2
enter the second number:2
The addition of 2 and 2 is : 4
The multiplication of 2 and 2 is : 4
The division of 2 and 2 is : 1.0
The modulus of 2 and 2 is : 0
The exponentiation of 2 and 2 is : 4
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

3. Write a program that prompts the user to enter a number and determines whether it is positive, even, positive odd, negative, or negative odd. Marks [2*5]

Output:

```
Desktop\fods1\q3.py"
enter a number:2
The number is positive even
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

```
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\Us
Desktop\fods1\q3.py"
enter a number:0
The number is zero
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

```
python -u
Desktop\fods1\q3.py"
enter a number:-2
The number is negative even
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

```
Desktop\fods1\q3.py"
enter a number:-1
The number is negative odd
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

```
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\
Desktop\fods1\q3.py"
enter a number:1
the number is positive odd
```

4. Write a program to take a single number as input and display: **Marks [2*5]**

- I. Cube of the number
- II. Cube root of the number
- III. Natural logarithm of the number
- IV. Base-2 logarithm of the number
- V. The number raised to the power of 6

Output:

```
● PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "
:\Users\userr\OneDrive\Desktop\fods1\q4.py"
enter a number :2
Cube : 8.0
Cube Root : 1.2599210498948732
Natural Log : 0.6926474305598223
Base-2 Log: 0.9992790131531383
Power of 6 : 64.0
○ PS C:\Users\userr\OneDrive\Desktop\fods1>
```

The program takes input as float and calculate, cube, cube root, natural log, base-2 log and power of 6 .

5. Write a program to input the marks of **6 subjects**. Calculate total, average, percentage, and grade according to the rules below: **Marks [2*5]**
- Distinction: 85% or above
 - First Division: 70% and above
 - Second Division: 55% and above
 - Third Division: 45% and above
 - Fail: Below 45%

Output:

```
python -u "c:\Users\userr\OneDrive\
Desktop\fods1\q5.py"
enter marks of subject 1 : 50
enter marks of subject 2 : 60
enter marks of subject 3 : 70
enter marks of subject 4 : 40
enter marks of subject 5 : 80
enter marks of subject 6 : 90
Total = 390.0
Average = 65.0
Percentage = 65.0
Grade = Second Division
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

Here, marks of six subject is entered and program displays, total average , percentage, and grade.

6. Write a program to display all **prime numbers within a user-defined range**. Additionally, print the **count of prime numbers** in that range along with their sum. **Marks [10]**

Output:

```

● Enter start number : 1
  Enter end number : 20
  prime numbers are :
  2
  3
  5
  7
  11
  13
  17
  19
  Count of a prime num is : 8
  The sum of prime num is : 77
○ PS C:\Users\userr\OneDrive\Desktop\fods1>

```

Here, program displays all prime numbers within a user defined range, and print sum and its count respectively.

7. Write a program to find all numbers between **1000 and 2500** (inclusive) that are divisible by 9 but not by 6. Extend the program so that the range limits (1000 and 2500) can also be entered by the user. **Marks [10]**

Output:

```

● PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\Users\userr\OneDrive\Desktop\fods1\q7.py"
  Enter start number : 1000
  Enter end number : 1200
  Number divisible by 9 but not by 6 :
  1017
  1035
  1053
  1071
  1089
  1107
  1125
  1143
  1161
  1179
  1197
  count : 11
  Sum : 12177
○ PS C:\Users\userr\OneDrive\Desktop\fods1>

```

8. Write a program to compute the factorial of a number entered by the user. Ensure the input is a valid positive integer. If the user enters anything else, display an error message such as **"Invalid input! Please enter a positive integer."** Marks [10]

Output:

```
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\Users\userr\OneDrive\Desktop\fods1\q8.py"
Enter a number : 5
Factorial of 5! is 120
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

The program print error message if user enter number other than positive integer,

```
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\Users\userr\OneDrive\Desktop\fods1\q8.py"
Enter a number : 5
Factorial of 5! is 120
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "c:\Users\userr\OneDrive\Desktop\fods1\q8.py"
Enter a number : -5
Invalid input! Please enter a positive integer.
PS C:\Users\userr\OneDrive\Desktop\fods1>
```

9. Write a menu-driven program that repeatedly asks the user for numbers and computes the **separate sums of positive and negative numbers**. Continue until the user chooses to exit the program. Marks [10]

Output:

```
--- Menu ---
1. Enter a number :
2. Show sums
3. Exit
Enter your choice : 
```

To enter a number,

```
Enter your choice : 1
Enter a number : 2
Added to positive Sum !
```

```
Enter your choice : 1
Enter a number : 2
Added to positive Sum !
```

```
Enter your choice : 1
Enter a number : -1
Added to negative sum !
```

To show sums,

```
Enter your choice : 2
Positive Sum : 4
Negative Sum : -1
```

The sum is four because user entered 2 twice.

To exit,

```
--- Menu ---
1. Enter a number :
2. Show sums
3. Exit
Enter your choice : 3
Bye Bye
PS C:\Users\userr\OneDrive\Desktop\fods1>
```


10. Write a number guessing game in Python with the following rules: **Marks [2 * 5 = 10]**

- The computer generates a random number between 1 and 50.
- The user must guess the number.
- After each guess, the program tells the user whether the guess is **too high, too low, or correct.**
- The user gets a maximum of **7 attempts.**
- If the user does not guess correctly within the attempts, display **"Better luck next time!"** and end the game.

Output:

```
EDITE (Desktop (10452 (q207p))
Welcome to Number Guessing Game!
I have picked a number between 1 and 50!
You have 7 attempts!

Attempts remaining: 7
Enter Your Guess: 
```

This program gives seven attempts to guess the number computer picked between 1 to 50.

```
PS C:\Users\userr\OneDrive\Desktop\fods1> python -u "C:\Users\userr\OneDrive\Desktop\fods1\q10.py"
Welcome to Number Guessing Game!
I have picked a number between 1 and 50!
You have 7 attempts!

    Attempts remaining: 7
Enter Your Guess: 10
Too Low!

    Attempts remaining: 6
Enter Your Guess: 30
Too Low!

    Attempts remaining: 5
Enter Your Guess: 40
Too Low!

    Attempts remaining: 4
Enter Your Guess: 45
Too High!

    Attempts remaining: 3
Enter Your Guess: 43
Too High!

    Attempts remaining: 2
Enter Your Guess: 42
Too High!

    Attempts remaining: 1
Enter Your Guess: 41
Correct! You guessed it in 7 attempts!
PS C:\Users\userr\OneDrive\Desktop\fods1> 
```